thread[1]. This is the point where we should question our design: is it really necessary to run these algorithms sequentially?

During the course of this chapter, we will answer this question, and present you with alternatives, design patterns and examples that will give you the tools to spot opportunities for code parallelization, and use the best practices in the industry to easily achieve major speedups.

After going through this chapter, readers will understand the difference between parallel and distributed computing, discover *canopy clustering*, and, learn about MapReduce, a computational model for distributed computing, and finally be able to re-write the clustering we have seen in the previous chapter to operate in a distributed environment.

13.1 Parallelization

Although the RAM model (presented in appendix B) is traditionally single-threaded and algorithm analysis usually focuses on sequential execution and improving the running time of single-process applications, parallelization, when applicable, can allow for tremendous speed-ups, and it should be in the tool belt of every software engineer.

When it comes to algorithm analysis in coding interviews, you might find out there are diverging opinions on this point. As a personal anecdote, during the same round of interviews I found two